

DSA 4030: BIG DATA SECURITY

END OF SEMESTER PRACTICAL GROUP PROJECT

SUMMER SEMESTER 2026

GROUP 3:

Kemo Dibassy ID: 669111

Angel Musomba ID: 667530

Valerian Murago ID: 669513

1. Executive Summary

1.1 Purpose

A hardened data-ingestion pipeline that lets five simulated clinic sites across Kenya push hourly patient-record batches into a central system, with every file authenticated, integrity-checked, encrypted, and audited before it becomes queryable data.

1.2 Why It Matters

Health records are highly sensitive; a single leaked field can expose a real patient, regulators require proof of who accessed what and when, and an attacker only needs one unverified upload to inject false data. The system answers three questions for every incoming file: is the sender genuine, has the file been altered, and can every action be proven after the fact.

1.3 Architecture: Five Gates Every File Passes Through

1. **Clinic Upload:** Hourly CSV batch arrives with a signed manifest
2. **Authenticate & Authorize:** Credentials and role permissions checked (auth.py)
3. **Verify Integrity:** SHA-256 hash + HMAC signature checked against the manifest
4. **Encrypt & Store:** Each record encrypted with Fernet (AES-128-CBC + HMAC)
5. **Audit Log:** Every attempt, success or rejection, written to database and file

A file failing steps 2 or 3 is rejected outright and quarantined, it never reaches encryption or storage.

1.4 Security Controls

Auth+RBAC: PBKDF2-HMAC-SHA256 (200,000 iterations); three roles (admin, clinic_uploader, auditor), each scoped so that even a valid clinic login can never decrypt patient data ,only admin holds that permission

Tamper Rejection: Any hash/signature mismatch is caught and quarantined before ingestion

Encryption at Rest: Every record encrypted individually; no plaintext PII on disk

Dual-Channel Audit Logging: A queryable SQLite table plus a human-readable append-only log

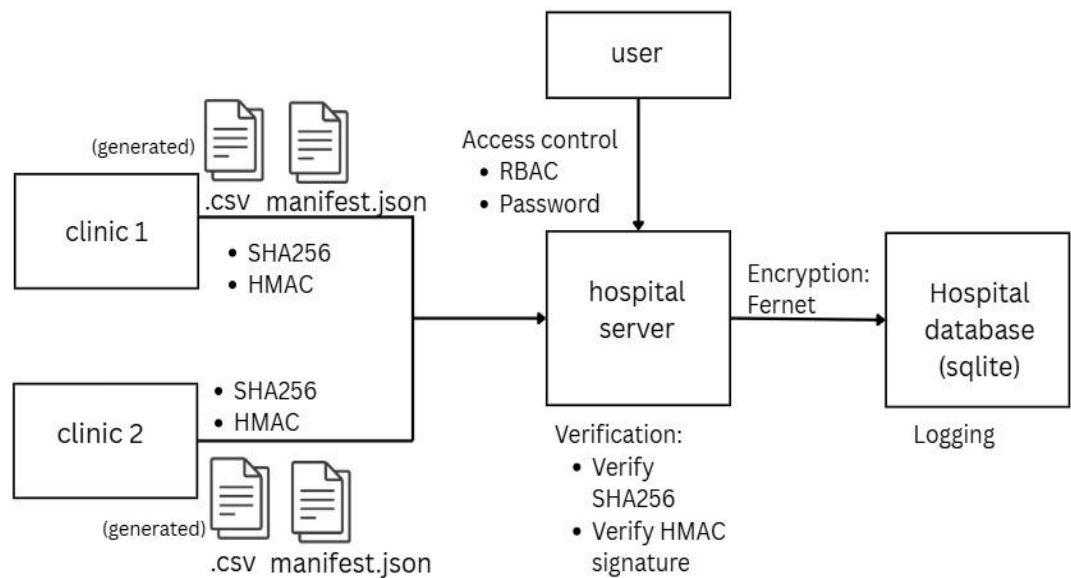
1.5 Validation: 6 / 6 Tests Passed

All six automated tests passed against the live system: tamper detection, end-to-end rejection, invalid-authentication handling, RBAC enforcement, encryption verification (0 of 200 scanned blobs leaked known plaintext terms), and audit completeness (13 successes, 1 rejection, all logged).

1.6 Bottom Line

This is a working, tested baseline for secure clinical data ingestion, the core controls (authentication, integrity, encryption, audit) are proven end-to-end; the next steps are about hardening it for production deployment rather than filling design gaps

2. System architecture diagram



3. System Implementation Report

3.1 Development Environment

The secure data ingestion system was implemented in a Python 3.10 environment, using SQLite as the storage backend together with a small set of standard and third-party libraries. SQLite was chosen for local development and testing because it requires no separate database server while still supporting the SQL operations, foreign keys, and transactional inserts needed for the project.

3.1.1. Software and Package Requirements

Component	Purpose
Python 3.10+	Application runtime
SQLite	Local storage of encrypted patient records, audit logs, and integrity data
cryptography	Fernet symmetric encryption and decryption
hashlib	SHA-256 hashing of files and records
hmac	Manifest signing and constant-time verification
csv	Reading generated clinic datasets
json	Manifest creation and parsing, and record serialization before encryption
sqlite3	Database connectivity from Python

3.2. Project Structure

The project separates dataset generation, authentication, and secure ingestion into distinct modules with data, logs, and cryptographic key material kept outside the source tree.

The file structure is as shown below:

```
project/
|
+-- data/
|   +-- incoming/
|   +-- manifests/
|   +-- quarantine/
|
+-- logs/
|   +-- ingestion.log
|
+-- storage/
|   +-- hospital.db
|   +-- encryption.key
|   +-- hmac.key
|
+-- src/
|   +-- generate_dataset.py
|   +-- secure_ingest.py
|   +-- auth.py
|   +-- simulate_uploads.py
|
+-- requirements.txt
```

The data/incoming directory holds clinic batch files awaiting ingestion. Files that fail integrity verification are moved to data/quarantine rather than being processed further. The data/manifests folder stores the signed SHA-256 and HMAC values recorded for each file at registration time in JSON format.

All cryptographic key material and the SQLite database itself live under storage, kept separate from the application source.

3.3. Dependency Installation

The only third-party dependency is the cryptography package, which provides the Fernet encryption implementation. It can be installed directly or via the project's requirements file:

```
pip install cryptography

# or

pip install -r requirements.txt
```

All other modules used (hashlib, hmac, csv, json, sqlite3) are part of the Python standard library and require no separate installation.

3.4. Database Configuration

The application uses a single SQLite database, storage/hospital.db, initialized by db_init.py. Four tables make up the schema:

	Purpose
users	Stores authentication credentials (passwords are both salted and PBKDF2-hashed). A role assigned to each account
patients	Stores encrypted patient records, together with a plaintext hash used for post-decryption integrity checking
file_integrity	Registry of SHA-256 hashes and HMAC signatures for every file registered by the ingestion pipeline
upload_log	Audit trail of every ingestion attempt, successful or not, including the actor, action, and outcome

The users table enforces role values for RBAC through a CHECK constraint restricting the role column to admin, clinic_uploader, or auditor. The patients table enforces a unique constraint on record_id, which is what allows duplicate records to be detected and rejected during ingestion rather than causing a clash or a duplicate row.

3.5. Security Key Configuration

Two cryptographic keys are generated automatically on first execution if they do not already exist, using a shared

helper function, `_load_or_create_key()`, which generates the key, writes it to disk, and restricts its file permissions to the owner only.

3.5.1 Encryption key: `storage/encryption.key`

A Fernet key used to encrypt and decrypt patient records before they are written to or read from the database. Fernet combines AES-128 in CBC mode with an HMAC for authentication, so encrypted records cannot be modified without detection.

3.5.2 HMAC key: `storage/hmac.key`

A 32-byte random secret used to sign and verify the manifest associated with each ingested file. Because this key is never transmitted or stored in the manifest itself, an attacker who modifies a file cannot also produce a valid HMAC for the altered content.

Both keys are generated once and reused on every subsequent run, ensuring that records encrypted or manifests signed in one session remain valid in later sessions.

3.6. Dataset Configuration

Synthetic datasets are produced by `generate_dataset.py`. This is a script that simulates hourly CSV uploads from five clinics: `Clinic_Nairobi_A`, `Clinic_Nairobi_B`, `Clinic_Mombasa_A`, `Clinic_Kisumu_A`, and `Clinic_Nakuru_A`.

Each record contains synthetic patient identifiers, demographics, diagnosis, department, vital signs, attending physician, and a visit timestamp. No real patient data is used at any point. An example of a filename produced by the generator is:

```
Clinic_Kisumu_A_20260718_0300.csv
```

The script generates 12 hourly batches, each 10,000 records, creating a dataset of 120,000 records. Each batch is written as a separate CSV file into `data/incoming`, named using the clinic identifier and the simulated timestamp, which allows the ingestion pipeline to process files in the order they would have arrived in a real system.

3.7. Manifest Generation

Before a file is ingested, it is registered using the `register_upload()` function in the `secure_ingest.py` script. This function represents the clinic side of the workflow. It involves confirming the file's integrity using the provided metadata before it is handed off for ingestion. The process:

1. The system computes a SHA-256 hash of the CSV file's raw bytes
2. An HMAC-SHA256 signature is generated that combines the filename and hash together, such that the manifest itself cannot be forged without the key (only accessible to authorized parties).
3. Writes a JSON manifest file to data/manifests containing the filename, hash, and signature
4. Writes a new row into the file_integrity table, recording the same values together with a registration timestamp

An example manifest entry:

```
{  
  "filename": "Clinic_Kisumu_A_20260718_0300.csv",  
  "sha256": "42cb9ff0...",  
  "hmac": "547f9cc4..."  
}
```

3.8. Authentication Configuration

System users authenticate with a username and password. The credentials are verified against the users table in the auth.py script. Throughout the whole pipeline, passwords are not stored in plaintext, to prevent access by unauthorized users. Instead, each password is salted (by adding a random 16-byte value to it) and hashed using PBKDF2-HMAC-SHA256 with 200,000 iterations. Authentication compares the hash of the submitted password against the stored hash rather than comparing passwords directly.

Role-based access control is implemented as a simple permission map, associating each role with the set of actions it is allowed to perform:

Role	Permissions
admin	upload, view_patients, view_logs, manage_users, decrypt
clinic_uploader	upload
auditor	view_logs

Three demo accounts are created automatically by db_init.py for testing each role: hospital_admin (admin), clinic_nairobi_a (clinic_uploader), and compliance_auditor (auditor). Any operation not present in a role's permission set is rejected by the require_permission() function, which raises an

AuthError naming the user, their role, and the action that was denied.

3.9. Secure Ingestion Configuration

Dataset ingestion is performed by running the `run_ingestion` script, which calls the `ingest_file()` function defined in `secure_ingest.py`. The pipeline consists of the following stages:

1. User authentication: the uploader's account credentials are verified
2. Role authorization: the account must hold the upload permission
3. Manifest verification: the file's current SHA-256 hash is recomputed and compared against the stored manifest
4. HMAC validation: the manifest's signature is recomputed and compared using `hmac.compare_digest`, which also takes a constant validation time, hence preventing timing attacks while enabling detection of a forged manifest
5. Quarantine of tampered files: any file that fails hash or HMAC verification is moved to `data/quarantine`, and the attempt is logged, rather than being ingested
6. Encryption of validated records: each row that passes verification is serialized to JSON, hashed, encrypted with Fernet, and stored
7. Storage in SQLite: the encrypted record, its plaintext hash, source filename, and ingestion timestamp are inserted into the `patients` table
8. Audit logging: every stage is recorded, including both failures and successes.

Only files that pass both the authentication check, the authorization check and the integrity verification step proceed to encryption and storage. Duplicate records, identified by a repeated `record_id`, are skipped during insertion rather than overwriting existing data.

3.10. Encryption Configuration

Validated patient records are encrypted using the Fernet implementation provided by the cryptography library, which combines AES-128 with HMAC-SHA256 for authenticated encryption. Each CSV row is converted to JSON, hashed with SHA-256, and encrypted; both the encrypted blob and the plaintext hash are stored in the `patients` table.

This plaintext hash supports integrity verification at read time in the function `decrypt_record()`, which requires the `decrypt` permission and is therefore restricted to the admin role. It decrypts the stored blob and recomputes the SHA-256 hash of the result, raising an error if the recomputed hash does not match

the value stored at ingestion time. This detects any corruption or tampering that might have occurred to the encrypted data while at rest.

3.11. Logging Configuration

Every ingestion attempt is audited. Logs are written to both the `upload_log` table in SQLite and the file `logs/ingestion.log`. Events recorded include:

1. Authentication failures due to invalid credentials
2. Authorization failures where permissions do not allow an operation
3. Integrity failures where there is a hash mismatch or invalid HMAC signature
4. Successful uploads, including the number of records ingested
5. Decryption requests performed by administrators

Each log entry records a timestamp, username, the action attempted, the filename involved, the resulting status, and any additional detail such as the specific reason for a rejection.

4. Testing Configuration

System testing is done using the `security_tests` script. The tests were all performed on the synthetic datasets, whose simulated ingestion is done through `simulate_uploads.py`, which registers and then ingests every file found in `data/incoming`. The following tests were used to validate the security controls. All tests conducted were passed.

1. **Test for tamper detection:** Checks whether the system detects a file that was altered after it was signed. This is tested by copying a registered clinic file, modifying one field's value, then recomputing its SHA-256 hash and calling `verify` against the manifest. It fails to pass if the function still returns `True` despite the content change
2. **Test for rejection at ingestion:** Checks whether a tampered file is blocked at the point of ingestion and never makes it into storage. Tested by calling `ingest_file()` on the same tampered file from Test 1 using valid clinic credentials, then querying the `patients` table for any record under that filename. It fails to pass if `ingest_file()` reports success, or if any row from that file is found in the database.
3. **Test for issue authentication:** Checks whether invalid credentials are correctly rejected. Tested by calling `authenticate()` with a real username but the wrong password, and also with a username that doesn't exist. It fails if either call does not raise an `Auth` error.

4. **Test for RBAC:** Checks whether record decryption is restricted to the admin role only. Tested by calling `decrypt_record()` for the same record using `clinic_uploader`, `auditor`, and `admin` roles. Fails if either `clinic_uploader` or `auditor` successfully decrypts the record instead of being denied, or if the admin call fails.
5. **Test for encryption at rest:** Checks whether stored patient data is actually unreadable without the encryption key. This is tested by reading the raw encrypted blobs in the `patients` table and scanning them for plaintext terms from the synthetic dataset (such as surnames & diagnoses). Fails to pass if any plaintext term is found inside an encrypted blob.
6. **Test for auditing:** Checks whether each ingestion attempt, whether successful or rejected, is captured in the audit trail. Tested by querying the `upload_log` table for successful and rejected status counts after running all the tests, then confirming the ingestion logs exist and contains valid entries. Fails to pass if there are zero success events, zero rejected events, or the log file is missing.

5. Evidence Portfolio

This section collects the supporting evidence: the project layout as seen in the IDE, the captured test-execution log, key configuration, and placeholders for terminal screenshots to be inserted from your own run.

5.1 Project structure (IDE view)

The screenshots below show the project tree in VS Code: the incoming clinic batches, generated manifests, the encrypted store (`hospital.db`), key files, logs, and the test artefacts.

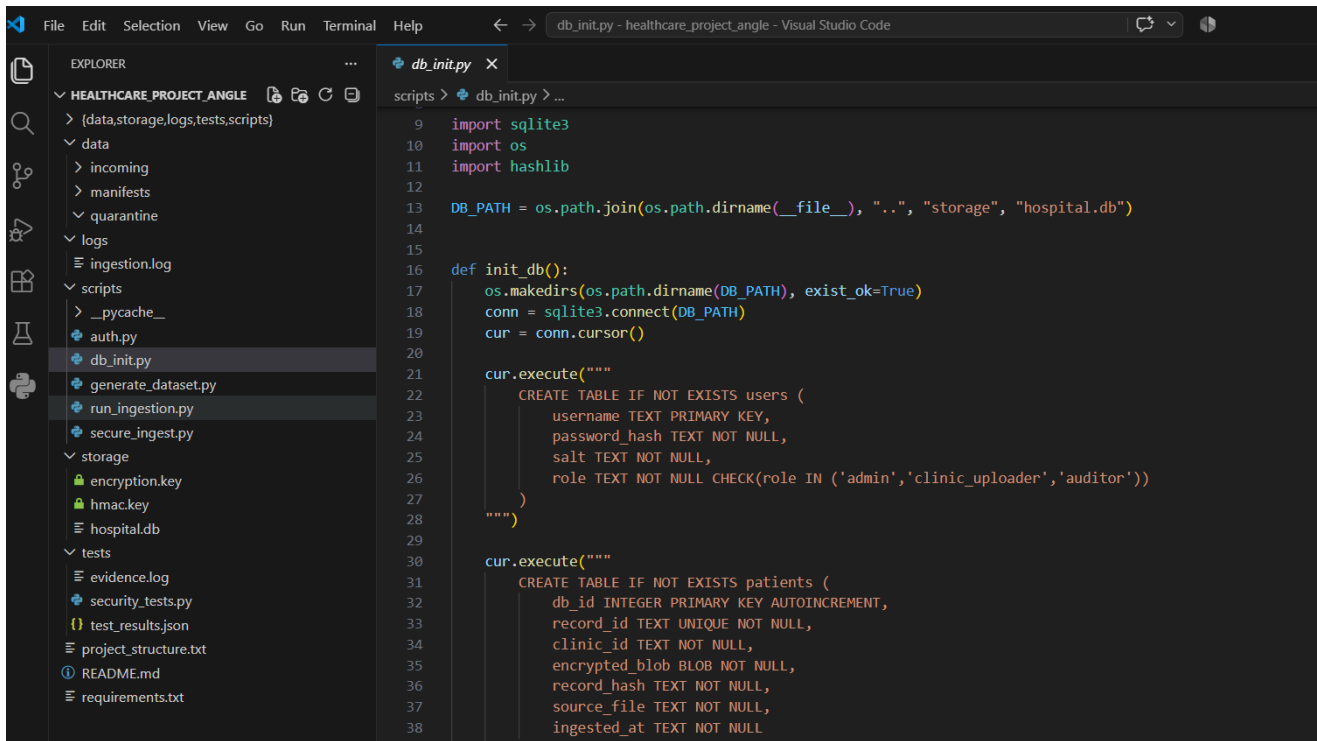


Figure 2 — Data, manifests, logs, script, storage keys, hospital.db, and the tests folder (evidence.log, security_tests.py, test_results.json).

5.2 Security test execution log (evidence.log)

The following is the captured output of the six-test run.

```

SECURITY TEST EXECUTION LOG
=====

=== TEST-1: PASS ===
Objective: Confirm the system detects and rejects a file altered after signing
Actual:  original_hash=561fla83def2e605... new_hash=c7fef8b17317cb2c...
        verify_result=False, detail='HASH MISMATCH: expected 561fla83..., got c7fef8b1...'

=== TEST-2: PASS ===
Objective: Confirm tampered files are rejected end-to-end and never reach storage
Actual:  ingest_file() result={'status': 'rejected', 'reason': 'HASH MISMATCH...'},
        records stored under this filename=0

=== TEST-3: PASS ===
Objective: Confirm the system rejects invalid credentials and unknown accounts
Actual:  wrong_password -> AuthError raised: True ('Invalid credentials');
        unknown_user -> rejected: True

=== TEST-4: PASS ===
Objective: Confirm RBAC restricts decryption to the admin role only
Actual:  clinic_uploader decrypt blocked=True, auditor decrypt blocked=True,
        admin decrypt succeeded=True

=== TEST-5: PASS ===
Objective: Confirm patient data is encrypted at rest (no plaintext PII in DB)

```

Actual: Scanned 200 encrypted blobs for 5 known plaintext terms. Matches found: 0

=== TEST-6: PASS ===

Objective: Confirm every upload attempt (success and rejected) is captured in the audit log

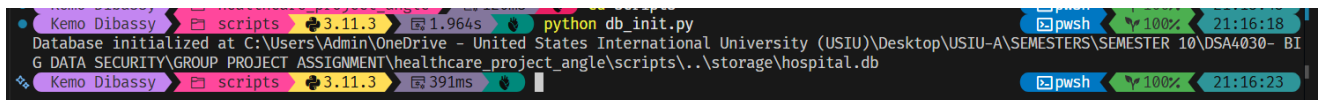
Actual: upload_log status breakdown={'REJECTED_TAMPER': 1, 'SUCCESS': 13};
logs/ingestion.log exists=True with 28 lines

SUMMARY 6/6 tests passed

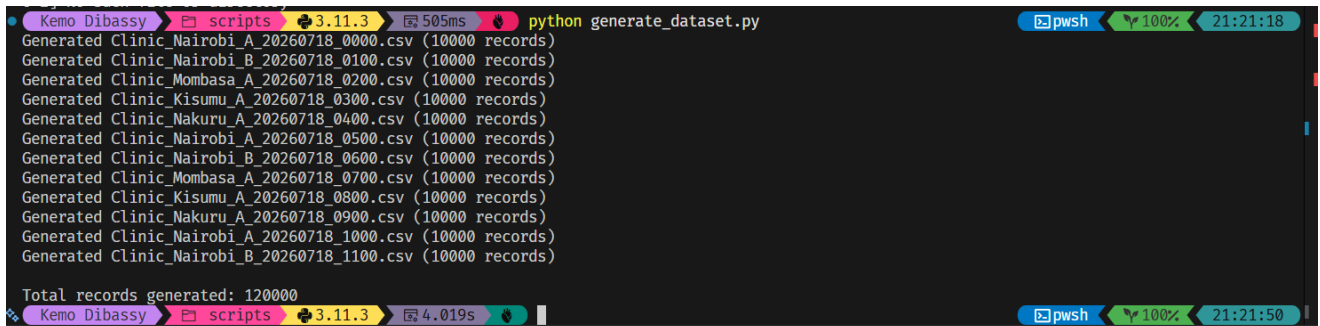
5.3 Terminal output

Run each stage and paste a screenshot of the terminal so the graders can see live output. Suggested captures:

- Screenshot below shows database initialization

A terminal window showing the execution of a Python script named db_init.py. The output indicates that the database was initialized at a specific path: C:\Users\Admin\OneDrive - United States International University (USIU)\Desktop\USIU-A\SEMESTERS\SEMESTER 10\DSA4030- BI G DATA SECURITY\GROUP PROJECT ASSIGNMENT\healthcare_project_angle\scripts\..\storage\hospital.db. The terminal also shows the user 'Kemo Dibassy' and the directory 'scripts'.

- Screenshot below shows generated dataset lines and total record count.

A terminal window showing the execution of a Python script named generate_dataset.py. The output lists 14 generated CSV files, each containing 10,000 records. The files are named after various clinics and locations, such as 'Generated Clinic_Nairobi_A_20260718_0000.csv (10000 records)'. At the bottom, it states 'Total records generated: 120000'. The terminal also shows the user 'Kemo Dibassy' and the directory 'scripts'.

- Screenshot below shows datasets ingested in manifests folder after hashing which will be used to check for integrity.

```
• Kemo Dibassy  scripts  3.11.3  224ms  python run_ingestion.py
Found 12 incoming batch files

[REGISTERED] Clinic_Kisumu_A_20260718_0300.csv
sha256=679cfa73eaae4dfedb1f2f6782dc2515d05c5e2b92757f46147a0f6b25cf3d8b
hmac=ee50eb6ab3ef88102ee9b81ae46e264f11cec5a7d97ac448c3abbbb57d260cfa
-> {'status': 'success', 'records_ingested': 10000}

[REGISTERED] Clinic_Kisumu_A_20260718_0800.csv
sha256=50b0d5bb6825916a4c3dfbd0c653ce2d0a9737dc58d9056b6f1e32b002828cde
hmac=ecd4a67032c1fd829e38068c44cc771865b5ab52db2024820394234d0b0f1486
-> {'status': 'success', 'records_ingested': 10000}

[REGISTERED] Clinic_Mombasa_A_20260718_0200.csv
sha256=def4154677412d6c8214d9603064c3b018044a72f8f7cd6adcefc74cba188f2b
hmac=cd7d559c8845361e98719b0b7d0eb974228800cf3a6d66893ebdfe248c61122b
-> {'status': 'success', 'records_ingested': 10000}

[REGISTERED] Clinic_Mombasa_A_20260718_0700.csv
sha256=375f6283aae9999f3e11e5f58ce18b77ce94917a88cc58a06993282cc81177c7
```

5.4 Configuration and key files

Secret keys are generated on first use and permission-restricted. Key configuration facts:

- encryption.key : Fernet key (AES-128-CBC + HMAC), file mode 0600.
- hmac.key: 32-byte random key used to sign manifests, file mode 0600.
- hospital.db: SQLite store containing users, patients, file_integrity, upload_log.

6. Risk Assessment

Risks are rated on Impact and Likelihood (Low / Medium / High). The residual controls column lists mitigations already implemented and those recommended for a production deployment.

ID	Risk	Impact	Likelihood	Recommended controls
R1	Encryption and HMAC keys stored on the same host as the data (local key files).	High	Medium	Implemented: 0600 permissions. Recommended: move to a KMS / HSM or OS keyring; rotate keys; separate key custody from data custody.

R2	No transport-layer encryption in the prototype hand-off (files assumed delivered locally).	High	Medium	Recommended: enforce TLS/HTTPS or SFTP for clinic→hospital transfer; mutual TLS to authenticate clinics.
R3	Shared clinic_uploader account reused across clinics (weak attribution).	Medium	High	Recommended: unique credentials per clinic/user; MFA; bind uploads to the authenticated clinic_id.
R4	Static, long-lived passwords; no logout or rate limiting on authentication.	Medium	Medium	Recommended: account logout / exponential backoff, password rotation, breach monitoring.
R5	SQLite single-file store — limited concurrency, no native at-rest DB encryption or backups.	Medium	Medium	Recommended: managed DB with TDE, automated encrypted backups, and point-in-time recovery.
R6	Audit log is append-only text but not cryptographically tamper-evident.	Medium	Low	Recommended: hash-chain or WORM/immutable log storage; ship logs to a separate SIEM.
R7	Fernet uses AES-128; some regimes mandate AES-256.	Low	Low	Recommended: adopt AES-256-GCM (e.g. via envelope encryption) where policy requires it.
R8	No malware/format validation on incoming CSV content beyond integrity.	Medium	Medium	Recommended: schema validation, size/row limits, and content scanning before ingestion.

R9	Keys and DB have no automated backup — loss of encryption.key means permanent data loss.	High	Low	Recommended: secure, tested key-escrow and backup procedures.
-----------	--	-------------	------------	---

7. Conclusion and Recommendations

7.1 Summary of outcomes

The project delivered a working secure-ingestion environment that satisfies the core security objectives: confidentiality (encryption at rest, RBAC), integrity (SHA-256 + HMAC verification with tamper rejection), and accountability (comprehensive audit logging). All six independent security tests passed against the live system, providing reproducible evidence that the controls behave as designed.

7.2 Lessons learned

- Layering matters: combining a content hash with an HMAC-signed manifest defeats both accidental corruption and deliberate forgery, whereas either control alone leaves a gap.
- Least privilege is easiest when enforced structurally: a permission allow-list per role made RBAC simple to reason about and to test.
- Evidence should be produced by the system, not asserted: having the test harness emit evidence.log and test_results.json made results reproducible and auditable.
- Key management is the hardest remaining problem; the cryptography is sound, but local key storage is the weakest link in the current prototype.

6.3 Recommended improvement

1. Move key material to a managed KMS/HSM or OS keyring, with rotation and separated custody (addresses R1, R9).
2. Add TLS/SFTP with mutual authentication for the clinic-to-hospital transfer (R2).
3. Issue unique per-user credentials with MFA and authentication lockout/rate-limiting (R3, R4).
4. Migrate from SQLite to a managed database with transparent data encryption and automated encrypted backups (R5).
5. Make the audit log tamper-evident via hash-chaining and forward it to a separate SIEM (R6).
6. Upgrade to AES-256-GCM where policy requires it, and add schema/content validation and scanning on ingest (R7, R8).

With these enhancements, the prototype can evolve into a production-grade, compliance-ready platform for secure healthcare data collection.

Appendices

Appendix A: Demo RBAC accounts

Username	Password	Role
hospital_admin	AdminPass!2026	admin
clinic_nairobi_a	ClinicUpload!99	clinic_uploader
compliance_auditor	AuditView!2026	auditor

Note: these are demonstration credentials for testing only and must be replaced before any real deployment.

Appendix B: Dataset description

The generator produces 120,000 synthetic records across 12 hourly batches from five clinics (Nairobi A/B, Mombasa A, Kisumu A, Nakuru A). Each record contains a UUID record_id, patient reference, clinic_id, name, DOB, gender, blood type, department, diagnosis, vitals (BP, heart rate, temperature), visit timestamp, attending physician, and notes. All values are randomly generated with a fixed seed (42) for reproducibility; no real patient data is used.

Appendix C: Code

The source code can be found in the **HEALTHCARE_PROJECT** folder attached to the document submission.